Information Security (CYBR3000)

Assignment — Semester 2, 2025

School of Electrical Engineering and Computer Science
The University of Queensland

Due 8th October 17:00 AEST

> One must learn by doing the thing; for though you think you know it, you have no certainty, until you try.
> — Sophocles

Revision 1.1.2

Overview This assignment provides practical experience in network security concepts. You will develop python3.13 code to simulate an "integrated network-border security appliance" — a combination firewall and intrusion detection system. No commercial organisation markets a product that is just a "firewall" any more. Modern commercial firewalls – typically with an "NG" somewhere in their name, for "Next-Generation" firewalls, compete on a wide range of advanced features well beyond simple IP/port filtering. They include built-in features like "Deep Packet" & application awareness (they inspect traffic by application in Layer 7, not just Layer 2 or Layer 3 IP and ports – and often with protocol validation to detect non-standard/malformed protocol use or even to re-write packets with the correct protocol to protect the inside hosts from malformed protocol attacks); "SSL Accelerators" [i.e. TLS] (dedicated TLS encryption/decryption engines to ensure all outbound traffic is encrypted/tunneled and to speed up inbound traffic decryption – this also means the firewall/IDS can perform inspection on the unencrypted payloads); signature-based and behavior-based intrusion detection systems and sometimes also AI/ML anomaly detection; Identity & "User Awareness" (policies including specific users or groups); and Quality of Service (QoS) prioritisation and bandwidth control based on any of these features.

Task Your simulated security appliance will only perform a small subset of these functions. It must accept simulated "IPv4 packets" (special format, described below), but may discard (drop), reject, forward, or process packets according to the rules provided below.

Your simulated security appliance will have **four** network interface cards (NICs), "*mgt*" (management interface), "*int*" (internal networks), "*dmz*" (demilitarised zone) and "*ext*" (external networks). You must write a class named "`Interface`" that includes the interface's MAC address, IP address, netmask, and default route; and must include the following methods:

- `def get_mac(self)`, that returns the MAC address of the interface as a string of six lowercase hexadecimal pairs separated by colons (e.g. "28:ee:52:85:f2:3a");
- `def set_mac(self, mac_address)`, that sets the interface MAC address as a string (as six lowercase hexadecimal pairs separated by colons);
- `def get_ip(self)`, that returns the IP address as a string of the IPv4 dotted quad;
- `def set_ip(self, ip_address)`, that sets the IP address as a string of the IPv4 dotted quad;
- `def get_mask(self)`, that returns the netmask as a string of the IPv4 dotted quad;
- `def set_mask(self, netmask)`, that sets the netmask as a string of the IPv4 dotted quad;
- `def get_default(self)`, that returns the default route IP address as a string of the IPv4 dotted quad;
- `def set_default(self, ip_address)`, that sets the default route IP address as a string of the IPv4 dotted quad; and
- `def send_packet(self, packet: bytes)`, that simulates sending a (special format, described below) "packet" (supplied as raw bytes) out that interface, by printing a message from the appropriate interface as "mgt|int|dmz|ext: sent packet " + the entire packet as a string in hexadecimal digits.

The Interface class must instantiate the **four** NICs. Interface MAC addresses and IP addresses/CIDR are:

- `mgt: 28:ee:52:85:f2:3a 192.168.96.23/20`;
- `int: 28:ee:52:e2:b7:30 10.0.0.1/16`;
- `dmz: 28:ee:52:4c:4d:70 10.1.0.1/24`;
- `ext: 28:ee:52:9c:61:ab 130.102.184.1/24`.

© The University of Queensland 2025

Your simulated `Interface` class instances will not actually simulate *receiving* "packets" to provide the frames to your simulation (*we are not programming hardware interrupts and your simulation will run as a single thread*)[1] instead you must develop an `InterfaceHandler` class to simulate the packet flows, including methods to:

- `def next_packet(self)`, read "packets" from a **special** (again PCAP-like, but not PCAP – special format – Do **NOT** use scapy) "`traffic.spcap`" (packet capture) file in the same directory as the code;
- `def send_packet(self, interface: str, packet: bytes)`, to "send out" packets via the appropriate interface's (referenced by the 3-character string mgt/int/dmz/ext) `send_packet` method; and
- `mgt_packet(self, packet: bytes)`, to be "actioned" by the simulated appliance itself, by printing a message "Actioned management packet " + the entire packet as a string in hexadecimal digits.

Your security appliance must perform port-address-translation (PAT) (also typically referred to generically as just NAT network-address-translation) on outgoing packets. This is also referred to as internal address hiding. The addresses on the packets are translated from the internal or DMZ address to the firewall's external address on a port not otherwise in use on the firewall. That way all traffic coming out of the organisation appears to be from the firewall itself and the replies are all returned to the firewall's external interface, providing no information about the internal networks. The firewall keeps track of the translated port numbers, so that when the destination replies, it can translate the reply back to the internal address and port number. Your security appliance must maintain a table of class `PatTable` that records the mappings from internal and DMZ hosts (IP address and port) to external ephemeral port numbers (ports 49152 – 65535). Your `PatTable` class must include the following methods:

- `def set_pat(self, in_address: str, in_port: int, out_port: int)`, adds the translation to the table and prints the message "NAT: allocate in_address:in_port -> out_address:out_port" – these are never removed in our simulated appliance – once set, the same combination of in_address and in_port will always be assigned the same out_port (in a real appliance these would need to be removed when no longer used otherwise the table would fill or all available ports would be allocated);
- `def get_unused_port(self)`, that returns a random ephemeral port number (ports 49152 – 65535) that is not already in the table;
- `def get_pat_in(self,out_port: int)`, that returns the IP:port as a string of the IPv4-dotted-quad:port;
- `def get_pat_out(self, in_address: str, in_port: int)`, that returns the external port number.

Finally your security appliance must perform security checks and either raise alerts (ALL alerts are printed messages, beginning with "ALERT drop: " immediately followed by a brief message stating why, as specified in the Rules) or route the packet it is processing to the correct interface (also with an accompanying printed message "ROUTE to int|dmz|ext", as appropriate). These actions are performed by the `PacketEngine` class along with the `RouteTable` class and `Connections` class. The `PacketEngine` class has to maintain a `RouteTable` as well as a list of current connections, including partly-opened connections. The `PacketEngine` class includes:

- `def process_packet(self, packet: bytes)`, that manages all the processing: performs security checks or routing the "packets" to the correct interfaces;
- `def check_packet(self, packet: bytes)`, that performs security checks (possibly raising alerts – printed messages); and
- `def route_packet(self, interface: str, packet: bytes)`, that simulates sending a packet to that interface for sending.

The `RouteTable` class includes methods to:

- `def resolve(self, ip)`, that returns the egress interface name for the destination IP address – longest-prefix match among local subnets, else 'ext' (default external).

The `Connections` class maintains a table of connections and includes methods (your methods must use the indicated types) to:

- `def add_or_update(self, nic: str, proto: int, src_ip: str, src_port: int, dst_ip: str, dst_p: int, state: str)`, for interface (mgt/int/dmz/ext), protocol, source address and port, destination address and port, the connection state ("new", "syn_sent", "established", "closed");
- `def state(self, nic: str, proto: int, src_ip: str, src_port: int, dst_ip: str, dst_p: int)`, that returns the connection state ("new", "syn_sent", "established", "closed") or null if no entry is found.

[1] EECS has a number of advanced networking and operating systems courses for those who want to learn about programming kernel modules.

© The University of Queensland 2025

Note our simulated security appliance does not maintain ARP tables or respond to ARP requests on any interface (that is just too much for this assignment).

The rules are as follows:

- Frames whose MAC destination is not this appliance or are malformed are already dropped. This is handled by the NIC hardware (system-on-a-chip) for you. You do *not* need to consider frame Preamble, Start Frame Delimiter, 802.1Q, or FCS (Frame Check Sequence) processing in your `PacketEngine`.
- Silently ignore (no alert) all URG and PSH TCP flags – process packets with these flags set as if the flags were not present (don't process data out of sequence/avoid URG-PSH-FIN attacks).
- Deny incoming ICMP ping requests (ICMP echo-request) of more than 64 bytes and alert with message "ALERT drop: oversize ping from <insert IP address> (<size> bytes)".
- Deny more than five incoming ICMP ping requests (ICMP echo-request) without at least five subsequent non-ping packets processed in the meantime and alert with message "ALERT drop: ping rate limit from <insert IP address>" (note that this particular simulation is structured this way is because we are not simulating varying time-of-arrival of packets at the interface – in reality a security appliance would be invoke a timer to check packet rates – our packets arrive as fast as we read them).
- Otherwise respond to (up to five at a time) each ICMP echo-request with the appropriate ICMP echo-reply on the appropriate interface.
- Drop all other ICMP traffic on all interfaces (a little extreme, but fine for our simulation) and alert with message "ALERT drop: ICMP type <insert type>:<insert code> not allowed by policy".
- Deny excessive incoming new TCP requests on any interface if there are more than 100 open incomplete connection requests by removing all partial connections from the table (again extreme, but fine for our simulation) and alert with message "ALERT drop: too many incomplete connections".
- Allow incoming SSH traffic (which is using TCP port 22) on the `mgt` interface from a specific IP address (192.168.96.9); and deny all other incoming TCP/UDP connections on the `mgt` interface and alert with message "ALERT drop: new incoming TCP not allowed by policy".
- Allow incoming DNS traffic (which is using UDP and TCP port 53) on `int` or `dmz` interfaces to outside (with PAT translation) and allow replies to those requests to return (via PAT). Do not respond to new inbound DNS queries on the `ext` interface (i.e. silently drop (no alert) inbound DNS queries on the `ext` interface).
- All incomming new TCP/UDP connections on the `ext` interface must be routed to proxies in the DMZ (i.e. HTTP/HTTPS to a webserver in the DMZ at 10.1.0.54, and an SSH jump box at 10.1.0.92).
- Allow SSH traffic (TCP port 22) on the `int` interface from internal IP addresses outbound via PAT to the external interface and allow replies to those connections from the `ext` interface via PAT back to the internal networks.
- Allow SSH traffic on the `int` interface from internal IP addresses to the DMZ (no PAT) and allow replies to those connections from the `dmz` interface back to the internal networks.
- Allow SSH traffic inbound on the `ext` interface routed to the DMZ SSH jump box via PAT and allow replies to those connections from the `dmz` interface (via PAT) back to the external networks.
- Allow SSH traffic from the jump box on the `dmz` interface to the to the internal networks (no PAT) and allow replies to those connections from the `int` interface back to the DMZ.
- Allow incoming HTTP (port 80) and HTTPS (port 443) on `ext` or `int` interfaces and allow replies to this traffic from `dmz` and `ext` only.
- Allow only established and related connections for any other incoming traffic, blocking any other new incoming traffic and alert with message "ALERT drop: new incoming TCP not allowed by policy".

© The University of Queensland 2025

Special Packet Capture Format – Do not use scapy.

SPCAP Format

First byte

1	0	1	0	1	0	Interface

TCP Frame

First Byte	Destination MAC address		Source MAC address		
Ethertype = 0x0080	IPprotocol	Source IP address	Destination IP address		Source TCP/UDP port
Dest TCP/UDP port	TCP Sequence		TCP Ack	Flags	TCP Payload
TCP payload (continued)					
TCP payload (continued) (variable length)					

UDP Frame

First Byte	Destination MAC address		Source MAC address	
Ethertype = 0x0080	IPprotocol	Source IP address	Destination IP address	Source TCP/UDP port
Dest TCP/UDP port	UDP Payload			
UDP payload (continued) (variable length)				

ICMP Frame

First Byte	Destination MAC address		Source MAC address		
Ethertype = 0x0080	IPprotocol	Source IP address	Destination IP address	ICMP Type	ICMP Code
ICMP payload (variable length)					

Figure 1: Diagram of the special packet capture format.

The first six bits are 101010.
The next two bits are the interface number:

- 00 = `mgt` (management),
- 01 = `int` (internal networks),
- 10 = `dmz` (demilitarised zone),
- 11 = `ext` (external networks).

This first byte is always followed by 48 bits (6 bytes) MAC Destination, then 48 bits (6bytes) MAC Source, Ethertype (2 bytes) is always 0x0800 (i.e. payload is always IPv4, never 0x86DD for IPv6),
Next is 8 bits (1 byte) IP protocol, 32 bits (4 bytes) IP Source, 32 bits (4bytes) IP Destination,
Then one of the following:

- TCP:
  - 16 bits TCP Source Port,
  - 16 bits TCP Destination Port,
  - 32 bits Sequence Number,
  - 32 bits Acknowledgement Number,
  - 8 bits TCP Flags:
    - CWR: Congestion window reduced,
    - ECE: ECN Congestion Experienced,
    - URG: Urgent,
    - ACK: Acknowledgment,
    - PSH: Push,
    - RST: Reset,
    - SYN: Synchronize,
    - FIN: Finish, and
  - TCP payload to end of record; OR
- UDP:
  - 16 bits UDP Source Port,
  - 16 bits UDP Destination Port, and
  - UDP payload to end of record; OR
- ICMP:
  - 8 bits ICMP Type,
  - 8 bits ICMP Code,
  - 32 bits ICMP header Content, and
  - ICMP payload to end of record.

These special frames have no Preamble, Start Frame Delimiter, 802.1Q, frame checksum (FCS). Our simulated security appliance ignores and does not process IPv4 options. Fragmentation is already handled and is not considered. All header checksums have already been verified and are not included in the packets.

© The University of Queensland 2025

Plagiarism. All work on this assignment is to be your own individual work. By submitting the assignment you are claiming it is your own work. You *may* discuss the overall general design of the application with other students. Describing details of how you implement your design with another student is considered to be **collusion** and will be counted as plagiarism.

Do **not** show other students your code. Do **not** look at other students' code. Do **not** copy fragments of code from other students. Code supplied by course staff (from *this* semester) is acceptable, but must be clearly acknowledged as described in the next paragraph. You do not need to acknowledge the provided code that was supplied with this specification – it is part of the specification and is not included in the academic integrity checks. Code generated by third-party tools is also acceptable, but **must** also be clearly acknowledged, see *Generative Artificial Intelligence* below.

You may find ideas of how to solve problems in the assignment through external resources (e.g. StackOverflow, textbooks, ...). If you use these ideas in designing your solution you ***must*** cite them. To cite a resource, provide the full bibliographic reference for the resource **in the block comment at the top of `appliance.py`**. For example:

```
"""
[1] E. W. Dijkstra, "Go To Statement Considered Harmful," _Communications of the ACM_,
    vol 11 no. 3, pp 147-148, Mar. 1968. Accessed: Aug. 6, 2025. [Online]. Available:
    https://www.cs.utexas.edu/users/EWD/transcriptions/EWD02xx/EWD215.html
[2] B. Liskov and J. V. Guttag, _Program development in Abstraction,
    specification, and object-oriented design_. Boston: Addison-Wesley, 2001.
[3] T. Hawtin, "String concatenation: concat() vs '+' operator," stackoverflow.com,
    Sep. 6, 2008. Accessed: Aug. 8, 2025. Available:
    https://stackoverflow.com/questions/47605/string-concatenation-concat-vs-operator
```

In the code where you use the idea, cite the reference in a comment. For example:

```
def method1():
    """
    What method1 does.
    [1] Used a method to avoid gotos in my logic.
    [2] Algorithm based on section 6.4.
    """

def method2():
    """
    What method2 does.
    """
    print("Some " + "content.") // [3] String concatenation using + operator.
```

You must be familiar with the university's policy on plagiarism.

`https://uq.mu/rl553`

If you have questions about what is acceptable, please ask course staff.

Generative Artificial Intelligence. Artificial Intelligence (AI) (e.g. ChatGPT or Copilot) and Machine Translation (MT) are emerging tools that may support students in completing this assessment task. Students may ***appropriately*** use AI and/or MT in completing this assessment task. Students ***must clearly reference*** any use of AI or MT ***in each instance***.

A failure to reference generative AI or MT use may constitute student misconduct under the Student Code of Conduct.

Please note that submitting an assignment that was written by AI, correctly referenced throughout, clearly identifying that all code was written by AI, is not plagiarism, but since the code was not written by the student, it ***has no academic merit*** and so would receive zero marks.

To pass this assessment, students may be required to demonstrate detailed comprehension of their submission independent of AI and MT tools, as part of the assignment marking process.

All submitted code will be subject to electronic plagiarism and collusion detection. The assignments are to be worked on individually and must be your own work except where the use of code written or provided by other entities is explicitly permitted by the assignment specification, ***and any such code is referenced in the manner required in the assignment specification.***

This is a learning exercise and you will harm your learning if you use AI tools inappropriately. Remember, you will be required to understand network security processes in the final exam.

Specification

This document specifies the assignment conditions and assessment. Read this assignment specification *carefully.*

- Implement the classes exactly as described.
- Read the specification carefully and understand the program specification ***before*** programming.
- Do not change the provided specification, ***including*** changing the specified names of classes, methods, or fields.
- You are encouraged to ***add*** additional methods or fields as you see fit – but the specified components **must** function exactly as specified.

Getting Started

To get started, read this document right to the end – do not start coding before you read this assignment specification – there are important assessment details following.

Next, download the provided framework from **BlackBoard**. Extract the archive into the directory you want to work from and open it with your preferred IDE.

Task

Fully implement a single python program, `appliance.py`, containing, at a minimum, each of the following, as described in the specification:

Interface class This class instantiates the each of the four NICs for the appliance.

InterfaceHandler class This class handles reading packets from a file and simulates sending or absorbing packets.

PatTable class This class handles port-address-translation (PAT).

PacketEngine class This class handles processing the packets.

RouteTable class This class handles the appliance's route table.

Connections class This class manages the appliance's current and half-open connections.

Project Overview

Remember, these are the **required** methods in each class. Implement these exactly as described. Do not change the provided specification in any way, ***including*** changing the specified names of classes, methods, or fields. You may ***add*** additional methods or fields – but the specified components **must** function exactly as specified.

Interface class This class instantiates the each of the four NICs for the appliance.

- `def get_mac(self)`, that returns the MAC address as a string of six lowercase hexadecimal pairs separated by colons (e.g. "28:ee:52:85:f2:3a");
- `def set_mac(self, mac_address)`, that sets the MAC address as a string (as six lowercase hexadecimal pairs separated by colons);
- `def get_ip(self)`, that returns the IP address as a string of the IPv4 dotted quad;
- `def set_ip(self, ip_address)`, that sets the IP address as a string of the IPv4 dotted quad;
- `def get_mask(self)`, that returns the netmask as a string of the IPv4 dotted quad;
- `def set_mask(self, netmask)`, that sets the netmask as a string of the IPv4 dotted quad;
- `def get_default(self)`, that returns the default route IP address as a string of the IPv4 dotted quad;
- `def set_default(self, ip_address)`, that sets the default route IP address as a string of the IPv4 dotted quad; and
- `def send_packet(self, packet: bytes)`, that simulates sending a "packet" (supplied as raw bytes) out that interface, by printing a message from the appropriate interface as "mgt|int|dmz|ext: sent packet " + the entire packet as a string in hexadecimal digits.

InterfaceHandler class This class handles reading packets from a file and simulates sending or absorbing packets.

- `def next_packet(self)`, read "packets" from a **special** (again PCAP-like, but not PCAP – special format – Do **NOT** use scapy)) "`traffic.spcap`" (packet capture) file in the same directory as the code;
- `def send_packet(self, interface: str, packet: bytes)`, to "send out" packets via the appropriate interface's (referenced by the 3-character string mgt/int/dmz/ext) `send_packet` method; and
- `mgt_packet(self, packet: bytes)`, to be "actioned" by the simulated appliance itself, by printing a message "Actioned management packet " + the "packet" as a string in hexadecimal digits.

PatTable class This class handles port-address-translation (PAT).

- `def set_pat(self, in_address: str, in_port: int, out_port: int)`, adds the PAT translation to the table – these are never removed in our simulated appliance – once set, the same combination of in_address and in_port will always be assigned the same out_port (in a real appliance these would need to be removed when no longer used otherwise the table would fill or all available ports would be allocated);
- `def get_unused_port(self)`, that returns a random ephemeral port number (ports 49152 – 65535) that is not already in the table;
- `def get_pat_in(self,out_port: int)`, that returns the IP:port as a string of the IPv4-dotted-quad:port;
- `def get_pat_out(self, in_address: str, in_port: int)`, that returns the external port number.

PacketEngine class This class handles processing the packets.

- `def process_packet(self, packet: bytes)`, that manages all the processing: performs security checks or routing the "packets" to the correct interfaces;
- `def check_packet(self, packet: bytes)`, that performs security checks (possibly raising alerts – printed messages); and
- `def route_packet(self, interface: str, packet: bytes)`, that simulates sending a packet to that interface for sending.

RouteTable class This class handles the appliance's route table.

- `def resolve(self, ip)`, that returns the egress interface name for the destination IP address – longest-prefix match among local subnets, else 'ext' (default external).

Connections class This class manages the appliance's current and half-open connections.

- `def add_or_update(self, nic: str, proto: int, src_ip: str, src_port: int, dst_ip: str, dst_p: int, state: str)`, for interface (mgt/int/dmz/ext), protocol, source address and port, destination address and port, the connection state ("new", "syn_sent", "established", "closed");
- `def state(self, nic: str, proto: int, src_ip: str, src_port: int, dst_ip: str, dst_p: int)`, that returns the connection state ("new", "syn_sent", "established", "closed") or null if no entry is found.

© The University of Queensland 2025

Grading

Your solution will be graded in Gradescope.

Automated Testing & Checking

Conformance and functionality grading will be done automatically in a Linux environment. The environment will not be running Windows, and neither *PyCharm* nor *Eclipse* (or any other IDE) will be involved. Python 3.13 will be used to execute your code. To prevent infinite loops, or malicious code, from slowing down Gradescope, any test that takes longer than 10 seconds to execute [2] will be killed and identified as failing.

IDEs like *PyCharm* provide code completion hints. When importing libraries they may suggest libraries that are not part of the standard library. These will ***not*** be available in the test environment and your code will ***not*** execute. There will be a few tests in Gradescope during the submission period these tests only exist to show if your code will run for at least some of the tests. When uploading your assignment to Gradescope, ***ensure*** that Gradescope shows that your submission was executed successfully.

Your code must run in the test environment.
If your submission does not execute, **you will receive no marks**.

There are marks for each of the functions stipulated, from setting/getting Interface variables, to reading .spcap files, to routing, NAT, and for each of the relevant rules and any associated alerts. As long as your submission doesn't throw an immediate error on every test, you will get marks for various tested functions that execute successfully.

The provided `traffic.spcap` and `sample_output.txt` files are only **SAMPLES** for you to use. They are **not** the assessment files. The assessment tests will use different files containing much more, and more complex mixes of, traffic packets to test your submission.

Submission

Submission is via Gradescope. Submit your code to Gradescope *early and often*. Gradescope will give you a ***very small*** amount feedback on your code every time you submit, but it is *not* a substitute for testing your code yourself. Gradescope will not execute the full functionality assessment before the assignment closes. You are expected to test that your code correctly follows the specification yourself. After the assignment closes (including any extensions and any late submissions), Gradescope will then execute the full functionality assessment and then all results will be made available in Gradescope and the raw assignment marks (before any late penalties) will be available in Blackboard. Any late penalties will be applied before final assignment marks are published in Blackboard.

What to Submit Your submission ***must*** have just one single file called `appliance.py`

Do *not* submit ***any*** other files (e.g., no IDE files).

Provided tests A ***very small*** number of the unit tests used for assessing *functionality* will be provided in Gradescope. These will be used to test that your submission runs, each time you upload it.

These are meant to provide you with an opportunity to receive feedback on whether your code runs in the Gradescope Python 3.13 test environment or not. Passing these few the provided unit tests does ***not*** guarantee that you will pass all the tests used for functionality grading.

[2] All tests should execute in a small fraction of a second. Any test taking longer than a second to execute indicates faulty logic or malicious code.

Assessment Policy

Late Submission You must submit your code ***before*** the deadline. Code that is submitted after the deadline will receive a late penalty as described in the course profile. The submission time is determined by the time recorded on the Gradescope server. A submission is not recorded as being received until uploading your files completes. Attempting to submit at the last minute may result in a late submission.

You may submit your assignment to Gradescope as many times as you wish before the due date. If a misconduct case is raised about your submission, a history of regular submissions to Gradescope, which demonstrates progress on your solution, could support your argument that the work was your own.

You are ***strongly*** encouraged to submit your assignment on time, or by your assigned revised deadline if the School of Electrical Engineering and Computer Science has granted you an extension.

Experience has demonstrated that most students who submit their assignments late lose more marks due to the late penalties than they gain by making improvements to their work.

Extensions If an unavoidable disruption occurs (e.g. illness, family crisis, etc.) you should consider applying for an extension. Please refer to the following page for further information.

`https://uq.mu/rl551`

Do not email the course coordinator or other course staff to request an extension. All requests for extensions must be made via my.UQ, ***before*** the submission deadline.

Re-Grading If an *administrative error* has been made in the grading of your assignment, please contact the course coordinator (cybr3000@uq.edu.au) to request this be fixed. For all other cases, please refer to the following page for further information.

`https://uq.mu/rl552`

© The University of Queensland 2025

Revision Log

Revision: 1.1.2

Any clarifications requiring additional documentation will be listed in this section of the task sheet detailing the additional material issued and also an announcement will be made on the course Blackboard site.

Revision 1.1.0 and 1.1.1

In the `Connections` class, the parameter for interface, originally 'if', is a reserved word, (the 'if' had been truncated from a longer abbreviation that was unclear at the time). We will use 'nic' instead (network interface card [mgt|int|dmz|ext]), as described above in this revision:

- `def add_or_update(self, nic: str, …`
- `def state(self, nic: str, …`

I removed the duplicate "ingress" parameter from the end of `add_or_update`'s parameter list that did exactly the same as the "nic" at the front of the list. It was redundant passing two parameters with the same information.

Also the first line was missing from the original `support.py` to import regular expressions. You can add "`import re`" as the very first line in `support.py` or just use the repackaged version of `support.py`.

Revision 1.1.2

From a request at the beginning of the lecture – added the interface detail to the message in the summary of `Interface::send_packet` on page 6, so that it matches the detail given on page 1. (Page 6 did not have all the detail from page 1.) That is:
`def send_packet(self, packet: bytes)`, that simulates sending a "packet" (supplied as raw bytes) out that interface, by printing a message ***from the appropriate interface as*** "**mgt|int|dmz|ext:** sent packet " + the entire packet as a string in hexadecimal digits.

© The University of Queensland 2025